



Tecnicatura Universitaria en Inteligencia Artificial

IA4.2 Procesamiento del lenguaje natural

TRABAJO PRÁCTICO FINAL

Alva, Máximo

2026

INTRODUCCIÓN	3
DISEÑO DE LA FUENTE DE DATOS	3
BASE DE DATOS VECTORIAL	3
BASE DE DATOS TABULAR	4
BASE DE DATOS DE GRAFOS	4
PROCESAMIENTO DE TEXTO	5
TEXT SPLITTER (MarkdownHeaderTextSplitter)	5
MODELO DE EMBEDDING	5
MODELO DE LENGUAJE (LLM)	6
CLASIFICADOR DE INTENCIÓN AVANZADO	6
CLASIFICADOR MACHINE LEARNING	6
CLASIFICADOR LLM	7
COMPARACIÓN	7
PIPELINE DE RECUPERACIÓN	7
ENRUTADOR	7
BÚSQUEDA HÍBRIDA	7
GENERACIÓN Y CONVERSACIÓN (RAG)	8
RESULTADOS DE EJECUCIÓN (RAG)	9
AGENTE INTELIGENTE REACT	12
RESULTADOS DE EJECUCIÓN (ReAct)	13
CONCLUSIONES	20

INTRODUCCIÓN

Este trabajo práctico detalla el diseño, desarrollo e implementación de un asistente virtual inteligente especializado en información de una empresa de electrodomésticos, a partir de sus fuentes de datos internas, basado en el paradigma RAG (Retrieval-Augmented Generation) y evolucionado hacia una arquitectura de agente autónomo ReAct (Reasoning and Acting). El objetivo principal de este sistema es interactuar con el usuario en lenguaje natural, comprender la intención profunda de su consulta y recuperar información de forma dinámica desde múltiples fuentes de datos corporativas antes de sintetizar una respuesta.

DISEÑO DE LA FUENTE DE DATOS

Para garantizar una recuperación precisa según el tipo de consulta del usuario, las fuentes de datos del dominio se dividieron en tres repositorios de información con estructuras y propósitos diferenciados.

BASE DE DATOS VECTORIAL

Se implementó el motor de bases de datos vectoriales [Milvus Lite](#). Dentro de esta base se decidió almacenar todo el texto no estructurado y/o semiestructurado de la empresa: manuales de los productos (archivos en formato Markdown) con datos como especificaciones técnicas, procedimientos de uso, etc., así como también las reseñas por parte de los usuarios (archivos en formato txt) con un encabezado donde se destacan el nombre del producto y el puntaje otorgado y debajo el texto con la opinión.

El esquema de la colección almacena vectores de alta dimensionalidad junto con metadatos cruciales para el filtrado híbrido: `id` (int), `vector` (float de 384 dimensiones), `text` (fragmento enriquecido), `categoria` (string) e `id_producto` (string). La métrica de similitud configurada es la Similitud del Coseno.

Este almacenamiento es imprescindible para las búsquedas semánticas. Permite encontrar respuestas basándose en el significado y el contexto de la consulta del

usuario, incluso si no utiliza las palabras exactas presentes en los manuales o reseñas.

BASE DE DATOS TABULAR

Se definió implementar una base de datos relacional utilizando [SQLite](#) en memoria (:memory:). En esta base se decidió almacenar todos los datos estructurados y transaccionales del negocio, los cuales fueron extraídos e importados desde archivos CSV. Esta información abarca el catálogo completo de productos (incluyendo precios, stock y características técnicas), el historial de ventas y vendedores, el estado del inventario por sucursales, el seguimiento de tickets de soporte y el registro de devoluciones. El modelo de datos definido consta de seis tablas relacionales que se encuentran interconectadas mediante claves primarias y foráneas ([id_producto](#), [id_venta](#) o [id_vendedor](#)), asegurando así una correcta normalización e integridad referencial de los registros.

Este tipo de almacenamiento resulta adecuado para gestionar datos cuantitativos, financieros y categóricos exactos. Para operar con esta base, se diseñó una interfaz de consulta mediante lenguaje natural. En este flujo, un Modelo de Lenguaje (LLM) especializado actúa como motor de traducción: recibe la pregunta del usuario, la convierte en una consulta SQL válida, la ejecuta sobre la base de datos y devuelve la tabla resultante. Esta arquitectura explota todo el potencial del estándar SQL, permitiendo al sistema realizar operaciones tales como filtrados precisos por rangos numéricos, agregaciones matemáticas (promedios, sumatorias), etc.

BASE DE DATOS DE GRAFOS

Se implementó el motor de bases de datos de grafos [GrafitoDB](#) para almacenar el conocimiento relacional y topológico del negocio en memoria. Específicamente, esta base mapea las asociaciones entre distintas entidades, como la agrupación de productos en categorías, la vinculación de preguntas frecuentes (FAQs) con productos específicos, y la clasificación temática de dichas FAQs, datos integrados a partir de los archivos: productos (CSV) y faqs (JSON).

El modelo de datos diseñado consta de cuatro tipos de nodos principales ([Producto](#), [Categoria](#), [FAQ](#), [TemaFAQ](#)) interconectados por tres tipos de aristas o relaciones

direccionales que dictan la semántica de la red: [PERTENECE_A](#), [TIENE_FAQ](#) y [TRATA SOBRE](#).

El almacenamiento en grafos resulta superior para resolver consultas que involucran múltiples saltos relacionales e inferencia de jerarquías (por ejemplo: "*¿Qué otros productos existen en la misma categoría que la licuadora P0013?*"). Para interactuar con esta red, se replicó la estrategia de traducción utilizada en la base tabular: un LLM interpreta la consulta en lenguaje natural del usuario y genera autónomamente la consulta en lenguaje Cypher correspondiente. Esta arquitectura permite al sistema navegar por las dependencias de los nodos con un costo computacional menor que el modelo tabular y descubrir conexiones profundas.

PROCESAMIENTO DE TEXTO

TEXT SPLITTER (MarkdownHeaderTextSplitter)

En lugar de fragmentar los manuales de forma arbitraria por cantidad de caracteres (como lo haría un `RecursiveCharacterTextSplitter` clásico), se optó por un particionado semántico basado en la estructura Markdown del documento.

Los manuales técnicos están estructurados en jerarquías ([# Título](#), [## Sección](#), [### Subsección](#)). Al usar [MarkdownHeaderTextSplitter](#), garantizamos que cada fragmento de texto mantenga el contexto lógico de la sección a la que pertenece. Además, el código inyecta metadatos en la cabecera de cada chunk para que el fragmento retenga información de su procedencia y no pierda el sentido semántico al estar aislado en el espacio vectorial. Adicionalmente, se aplicó limpieza mediante expresiones regulares para remover marcas de formato innecesarias (asteriscos, guiones bajos) que podrían añadir ruido espacial al vector.

MODELO DE EMBEDDING

Seleccioné el modelo [paraphrase-multilingual-MiniLM-L12-v2](#) para realizar los embeddings. Al ser un asistente que recibirá consultas en español, es fundamental utilizar un modelo multilingüe entrenado para comprender la semántica con alta precisión. La arquitectura MiniLM produce vectores de 384 dimensiones. Esto reduce significativamente el uso de memoria RAM y el costo computacional al

cargar los datos en Milvus Lite y al calcular la distancia del coseno, en comparación con otros modelos de mayor tamaño. Al estar optimizado para tareas de paráfrasis, es ideal para sistemas QA, ya que empareja muy bien consultas informales de usuarios con textos técnicos de manuales que significan lo mismo pero están escritos de forma diferente.

MODELO DE LENGUAJE (LLM)

Para el núcleo de razonamiento del asistente virtual y la orquestación de herramientas, finalmente se decidió utilizar el modelo [Qwen 2.5 3B](#) de Alibaba Cloud, desplegado de forma local mediante [Ollama](#). Esta decisión se tomó luego de evaluar modelos alternativos como [Llama 3.2 3B](#) y [Gemma 3 1B](#). En primera instancia, estos dos modelos tendían a sufrir de sobrecarga de contexto al recibir grandes volúmenes de texto de las bases de datos, lo que ocasionaba que rompieran los formatos, que realizaran consultas SQL/Cypher erróneas o que alucinaran respuestas. Por su parte Qwen 2.5 3B demostró una estabilidad superior en tareas de razonamiento paso a paso, seguimiento estricto de plantillas (como el framework ReAct) y el uso autónomo de herramientas.

CLASIFICADOR DE INTENCIÓN AVANZADO

Para optimizar el rendimiento del sistema, se diseñó e implementó un clasificador de intención, a utilizar en la capa de enrutamiento previa al razonamiento del agente principal. Este clasificador tiene la responsabilidad exclusiva de analizar la pregunta del usuario y determinar qué motor de base de datos (tabular, grafos o vectorial) es el adecuado para resolverla. Para resolver esta tarea, se evaluaron e implementaron dos enfoques arquitectónicos distintos.

CLASIFICADOR MACHINE LEARNING

Se entrenó un modelo predictivo ligero de regresión logística utilizando un dataset sintético etiquetado con ejemplos de consultas. Para la extracción de características, se reutilizó el modelo de embeddings paraphrase-multilingual-MiniLM-L12-v2 empleado en la base vectorial. Esto asegura coherencia semántica en todo el pipeline y permite una rápida clasificación.

CLASIFICADOR LLM

Se configuró un modelo de lenguaje especializado mediante un System Prompt estricto que incluye ejemplos representativos para cada intención (Few-Shot). El LLM evalúa la semántica profunda de la consulta en tiempo real y devuelve únicamente la etiqueta de la base de datos a consultar.

COMPARACIÓN

Durante las pruebas de validación de ambas alternativas, se observó que tanto el clasificador de Machine Learning (Regresión Logística) como el clasificador semántico (LLM) arrojaron resultados idénticos y una precisión altamente satisfactoria. Dada la escala actual y la cantidad limitada de datos de prueba disponibles, ambos enfoques demostraron ser capaces de interpretar la intención del usuario y derivar la consulta a la base de datos correcta sin presentar errores mayores de clasificación. Ante esta paridad, se tomó la decisión de optar por el enrutador basado en LLM como la solución definitiva para el sistema.

PIPELINE DE RECUPERACIÓN

ENRUTADOR

El ciclo de vida de una consulta sigue un pipeline estructurado para garantizar respuestas precisas y eficientes. Una vez que el usuario ingresa su pregunta, el pipeline llama al clasificador LLM para detectar la intención del usuario y según esta, derivarlo a las interfaz del motor de bases de datos que le corresponda.

BÚSQUEDA HÍBRIDA

Cuando el enrutador determina que la consulta debe resolverse en la base de datos vectorial, no se realiza una búsqueda por similitud plana, sino que se ejecuta una búsqueda híbrida con reranking.

El proceso se ejecuta en cuatro pasos secuenciales:

1. **Búsqueda Semántica:** Se consulta la base de datos vectorial utilizando los embeddings generados a partir de la pregunta del usuario. Esto permite recuperar documentos basándose en lo conceptual y el significado, mitigando la barrera de los sinónimos.
2. **Buscador BM25:** En paralelo, se ejecuta una búsqueda tradicional basada en palabras clave utilizando el algoritmo [BM25Okapi](#). Este paso es clave para capturar coincidencias exactas que los modelos vectoriales suelen perder, como códigos de productos (ej. "P0017") o jerga técnica específica.
3. **Fusión:** Se unen los candidatos obtenidos de ambas estrategias. Para evitar redundancias en el contexto, se implementa un mecanismo de deduplicación que evalúa los primeros 50 tokens de cada fragmento, filtrando aquellos que ya ingresaron por otra vía, garantizando un conjunto de candidatos únicos.
4. **Reranking:** Al conjunto final se lo procesa mediante un [CrossEncoder](#). A diferencia de la vectorización clásica, este modelo toma como entrada el par (pregunta, documento), evaluando la interacción profunda y el nivel de atención cruzada entre ambos. Este modelo finalmente asigna un nuevo puntaje de relevancia absoluta y devuelve los documentos ordenados de mayor a menor.

Al implementar esta arquitectura de búsqueda híbrida, el sistema obtiene "lo mejor de ambos mundos", la recuperación paralela maximiza la exhaustividad (garantizando que no se escape información vital), mientras que el Reranking asegura una alta precisión en los primeros puestos.

GENERACIÓN Y CONVERSACIÓN (RAG)

La etapa final del RAG corresponde a integrar todos los componentes en un bucle conversacional gestionado por la clase [AsistenteVirtual](#). Este módulo se encarga de llamar al pipeline anterior para obtener la intención de la pregunta del usuario y los resultados de la base de datos correspondiente. Luego, formatea los resultados a texto legible por el LLM, valida los datos para filtrar casos donde no obtuvimos resultados y finalmente le provee el contexto con memoria del historial de preguntas al LLM para que genere una respuesta final.

Se implementaron las siguientes decisiones de arquitectura:

1. **LLM:** Se configuró el modelo con una temperatura nula ($\text{temperature}=0.0$) minimizando la creatividad estocástica del LLM, forzándolo a generar respuestas deterministas. Mediante un *System Prompt* estricto, se le impone la regla de anclar su conocimiento única y exclusivamente en el contexto provisto en cada turno, erradicando la posibilidad de que responda utilizando su conocimiento pre-entrenado.
2. **Formateo:** Dado que el LLM opera con texto plano, se diseñó un método de parseo que traduce las estructuras de datos crudas provenientes de los tres motores. Por ejemplo, si el enrutador derivó la consulta a la base tabular, DataFrame resultante se convierte en una tabla formato Markdown.
3. **Early Exit:** Se implementó una validación crítica antes de invocar al LLM generador. Si el contexto formateado indica que "No hay datos" o contiene un "Error" de ejecución, el sistema interrumpe el flujo y devuelve una respuesta predefinida disculpándose con el usuario. Esta decisión técnica ahorra recursos de cómputo (evitando una inferencia innecesaria del LLM) y reduce a cero la probabilidad de que el modelo alucine una respuesta para complacer al usuario ante la falta de información.
4. **Memoria Conversacional Escalable:** El asistente mantiene el hilo de la conversación utilizando las estructuras de memoria de LangChain. El contexto masivo recuperado se inyecta únicamente en el turno actual (acompañando a la pregunta), pero no se arrastra crudo a la memoria a largo plazo. Esto previene que la ventana de contexto del LLM se sature tras múltiples interacciones, manteniendo el historial ligero y focalizado en la conversación.

RESULTADOS DE EJECUCIÓN (RAG)

Para validar la arquitectura, se realizó una sesión de pruebas. A continuación, se presenta un análisis de la interacción, destacando los aciertos técnicos y las limitaciones estructurales encontradas.

 USUARIO: ¿Cómo uso mi licuadora para hacer smoothies?

 ASISTENTE: Para preparar un smoothie de frutas utilizando tu compacta licuadora ChefMaster P0004, sigue estos pasos:

1. Lava y corta las frutas en trozos medianos (2-3 cm).
2. Coloca los ingredientes líquidos primero (leche, yogurt, jugo) en la jarra.
3. Agrega las frutas y hielo a continuación.
4. Cierra herméticamente la tapa de la licuadora.
5. Comienza el procesamiento en velocidad baja (niveles 1-2) durante 10 segundos.
6. Aumenta gradualmente a velocidad alta (niveles 4-5).
7. Procesa durante 45-60 segundos hasta obtener una textura homogénea.
8. Si es necesario, usa la función PULSE para romper trozos grandes.
9. Verifica la consistencia y procesa adicionalmente 10-15 segundos si es necesario.
10. Apaga y desconecta la licuadora antes de retirar la jarra.
11. Sirve inmediatamente para obtener el mejor sabor y textura.

Recuerda que este procedimiento es para la licuadora ChefMaster P0004. Si tienes alguna otra modelo, como el P0007 mencionado en las reseñas, puede haber diferencias en su funcionamiento o características.

En la primera interacción, el enrutador derivó correctamente la consulta a la base vectorial. El sistema recuperó con éxito un manual de instrucciones y el LLM logró sintetizar un procedimiento de 11 pasos perfectamente estructurado.

 USUARIO: ¿Cuáles son las licuadoras de menos de \$200?

 ASISTENTE: Lo siento, no he encontrado información relevante sobre eso en nuestra base de datos. ¿Podrías intentar reformular tu pregunta o consultar otro tema?

En la segunda consulta, el sistema no encontró datos. En lugar de inventar precios para complacer al usuario, el mecanismo de Early exit interceptó el contexto vacío y disparó el mensaje predeterminado. Esto valida la robustez del freno de seguridad. Sin embargo, el análisis de los registros demostró que estos casos de "resultados no hallados" en la base tabular muchas veces se deben a consultas SQL sintácticamente correctas, pero semánticamente rígidas (por ejemplo, buscar con igualdades estrictas en lugar de utilizar comodines léxicos). Al operar linealmente, el sistema se rinde tras el primer intento fallido, evidenciando la limitación estructural de la arquitectura RAG y su incapacidad para reflexionar sobre su propio error y reformular la consulta.

 USUARIO: Quiero una con buenas reseñas

 ASISTENTE: Basándonos en las reseñas proporcionadas, te recomendaría la ****Yogurtera P0134**** y el ****Caloventor P0174****, ya que ambos productos han recibido calificaciones positivas de 4/5 estrellas.

- ****Yogurtera P0134****: Los usuarios lo describen como robusto, confiable y muy contentos con la compra.
- ****Caloventor P0174****: Se considera de primera calidad, eficiente y compacto, con una buena experiencia general.

Ambos productos parecen tener buenas reseñas y podrían ser una buena opción para ti.

En esta consulta, el usuario hace referencia explícita al sujeto de la conversación anterior (la licuadora). El motor de embeddings vectoriza la pregunta literal del usuario ("*Quiero una con buenas reseñas*") de forma aislada, perdiendo el sujeto tácito. Al buscar en Milvus Lite, el sistema trajo las reseñas con el puntaje más alto (Yogurtera/Caloventor), ignorando que el usuario seguía hablando de licuadoras.

 USUARIO: Quiero una licuadora con buenas reseñas

 ASISTENTE: Basándonos en las reseñas proporcionadas, te recomendaría el ****Licuadora P0006****, ya que ha recibido una calificación de 5/5 estrellas y una excelente reseña:

- ****Reseña positiva para Licuadora P0006****: "Quedé fascinado/a con este producto. Es muy versátil, práctico y preciso. Mi familia está muy contenta con la compra."

Aunque las otras licuadoras (P0006 y Ultra Licuadora P0008) también tienen reseñas mixtas, el Licuadora P0006 ha sido calificado de manera más positiva por un usuario.

Espero que esta recomendación te sea útil. Si tienes alguna otra pregunta o necesitas más información, no dudes en decírmelo.

En esta consulta, al explicitar el sujeto, el sistema sí logró recuperar el documento correcto.

 USUARIO: ¿Qué productos están relacionados con la categoría Cocina?

 ASISTENTE: Basándonos en el contexto proporcionado, los productos relacionados con la categoría de Cocina incluyen:

1. ****Licuadora****
2. ****Plus Licuadora Pro****
3. ****Compacto Licuadora****
4. ****Ultra Licuadora****
5. ****Procesadora****
6. ****Deluxe Procesadora****
7. ****Profesional Procesadora 2024****
8. ****Advanced Procesadora II****
9. ****Yogurtera****
10. ****Profesional Yogurtera****

11. **Smart Yogurtera**
12. **Advanced Yogurtera**
13. **Deluxe Yogurtera 3000**
14. **Advanced Deshidratador**
15. **Profesional Deshidratador**
16. **Elite Deshidratador**
17. **Ultra Deshidratador**
18. **Turbo Deshidratador**

Estos productos están directamente relacionados con la categoría de Cocina, ya que son electrodomésticos utilizados en el proceso de preparación y manipulación de alimentos.

En la quinta consulta, el sistema demostró un funcionamiento impecable del motor de grafos. El enrutador detectó la intención topológica, el LLM tradujo la consulta a Cypher correctamente y recuperó una lista exhaustiva de 18 productos interconectados con ese nodo, formateándolos de manera prolija.

AGENTE INTELIGENTE REACT

Se evolucionó la arquitectura a un agente ReAct. En este paradigma, el modelo de lenguaje no solo genera texto, sino que razona paso a paso y decide qué herramientas usar, evaluando las observaciones obtenidas antes de emitir una respuesta final.

La implementación de la clase [AsistenteVirtualReAct](#) incorpora las siguientes mecánicas de control y seguridad:

- **Orquestación de herramientas:** El agente dispone de un diccionario que mapea las distintas herramientas: [table_search](#) (SQL), [graph_search](#) (Cypher), [doc_search](#) (Vectorial) y [analytics_tool](#) (Gráficos). El agente decide dinámicamente cuál invocar basándose en la intención de la consulta.
- **Memoria de corto plazo anti-repetición:** Una limitación con la que me encontré fue que el agente era propenso a entrar en bucles infinitos repitiendo la misma acción fallida. Para solucionarlo, se implementó un registro temporal. Si el agente intenta invocar la misma herramienta con los mismos parámetros por segunda vez, el sistema bloquea la ejecución en la base de datos y le inyecta un error explícito, forzándolo a cambiar su estrategia.

- **Frenos de seguridad:** Se diseñó un mecanismo de control sobre el bucle iterativo para evitar el colapso cognitivo y el consumo excesivo de recursos:
 - **Context overflow:** Si una herramienta devuelve una observación masiva que supera el tope de caracteres, el sistema aborta el ciclo para proteger la ventana de contexto del LLM y solicita al usuario mayor especificidad.
 - **Falta de datos:** Si tras dos iteraciones el modelo sigue recibiendo mensajes de error o tablas vacías, el bucle se corta amablemente.
 - **Límite de razonamiento:** Se impone un tope de tres iteraciones de pensamiento. Si no llega a una conclusión, el sistema fuerza la salida.
- **Impulsos cognitivos:** Para asistir al modelo en el seguimiento estricto del formato ReAct, se inyectan sugerencias en el prompt de forma dinámica. Si el agente recupera información útil real, Python inyecta automáticamente la frase: *"Thought: ¡Excelente! La herramienta me devolvió la información..."* *\nFinal Answer: "*. Esto guía al modelo hacia la síntesis, evitando que se distraiga intentando realizar búsquedas adicionales innecesarias.

RESULTADOS DE EJECUCIÓN (ReAct)

Para evaluar la evolución de la arquitectura, se realizó una serie de pruebas con el asistente virtual utilizando el paradigma ReAct, donde el modelo tiene autonomía para utilizar herramientas y mantener el hilo de la conversación. A continuación, se presenta un análisis de la interacción.

 USUARIO: ¿Cómo uso mi licuadora para hacer smoothies?

 [PENSANDO] -> Para responder a esta pregunta, necesito buscar instrucciones de usuario y posibles preguntas frecuentes relacionadas con el uso de una licuadora.

 [USANDO]: 'doc_search' -> INPUT: 'instrucciones licuadora'

 [RESULTADOS]: Producto: Compacto Licuadora P0004 Marca: ChefMaster | Manual Técnico - Compacto Licuadora > Especificaciones Técnicas | - Modelo: P0004

- Nombre Comercial: Compacto Licuadora
- Categoría: Cocina - Preparación
- Marca: ChefMaster
- Color: Rosa
- Potencia: 1000W

- Capacidad: 2.0L
- Voltaje: 220V
- Peso Neto: 48.7 kg
- Garantía: 24 meses
- Certificaciones: CE, RoHS, ISO 9001
- Clase Energética: A++
- Origen: Importado

Producto: Compacto Licuadora P0004 Marca: ChefMaster | Manual Técnico - Comp
... [Texto truncado]

⚠ [FALLBACK]: Se asume salida como final.

🤖 ASISTENTE: Para hacer smoothies con tu licuadora ChefMaster P0004, sigue estos pasos:

1. ****Preparación****: Asegúrate de que todos los ingredientes estén a temperatura ambiente para obtener mejores resultados.
2. ****Limpieza y Verificación****: Antes de usar la licuadora, asegúrate de que no haya residuos del uso anterior. Lava el vaso y las piezas internas con agua tibia y jabón suave. Seca bien antes de usarla.
3. ****Montaje****: Coloca los ingredientes en el vaso de la licuadora. Comienza por los sólidos, luego añade líquidos como jugo o leche. Asegúrate de que no sobrepases la capacidad máxima del vaso (2.0L).
4. ****Seguridad****: Antes de encender la licuadora, asegúrate de que la tapa esté correctamente en su lugar y el sistema anti-derrames esté activo para evitar derrames o salpicaduras.
5. ****Uso****: Enciende la licuadora y ajusta al nivel deseado (generalmente se puede encontrar un botón o una palanca para controlar la velocidad). Deja que la licuadora trabaje durante unos minutos hasta que los ingredientes estén bien mezclados y sueltos.
6. ****Limpieza Post-Usos****: Una vez terminado, apaga inmediatamente la licuadora y retira el vaso para limpiarlo adecuadamente. No intentes desmontar las piezas internas mientras aún esté caliente.
7. ****Accesorios Opcionales****: Si tienes accesorios como una espátula de silicona, puedes usarla para ayudar a mezclar los ingredientes o retirar residuos del vaso.

Recuerda siempre seguir las instrucciones específicas del manual y no exceder la capacidad máxima del vaso. ¡Disfruta preparando tus smoothies!

Si tienes más preguntas o necesitas ayuda con algo más, no dudes en decírmelo.

Al igual que el asistente RAG, este logró hallar el manual de uso en la base vectorial con las instrucciones para hacer el smoothie. A diferencia del anterior, este redactó un nuevo paso a paso adaptado en base al hallado.

 USUARIO: ¿Cuáles son las cafeteras de menos de \$1000?

 [USANDO]: 'table_search' -> INPUT: 'cafeteras menores a 1000'

 [RESULTADOS]: [SQL Ejecutado: SELECT * FROM productos WHERE nombre LIKE '%cafetera%' AND precio_usd < 1000;]

Datos: [{ 'id_producto': 'P0122', 'nombre': 'Cafetera', 'categoria': 'Cocina', 'subcategoria': 'Pequeños Electrodomésticos', 'marca': 'ChefMaster', 'precio_usd': 173.03, 'stock': 20, 'color': 'Azul', 'potencia_w': 450.0, 'capacidad': '1.0L', 'voltaje': '220V', 'peso_kg': 8.3, 'garantia_meses': 36, 'descripcion': 'Presentamos la Cafetera de ChefMaster, un producto excepcional de la categoría Cocina. Con ... [Texto truncado]

 [FALLBACK]: Se asume salida como final.

 ASISTENTE: ¡Perfecto, encontramos una cafetera que se ajusta a tu presupuesto!

La opción que te recomiendo es la **Cafetera de ChefMaster** (ID Producto: P0122). Esta cafetera tiene las siguientes características:

- **Precio**: \$173.03
- **Capacidad**: 1.0L
- **Potencia**: 450W
- **Color**: Azul

Esta cafetera es robusta y de alta calidad, con una garantía de 36 meses respaldada por ChefMaster. Además, cuenta con tecnología vanguardista que facilitará tu vida diaria.

Si tienes alguna otra pregunta o necesitas más opciones, no dudes en decírmelo. ¡Espero que esta información te sea útil!

Ante esta consulta, el agente pivota hacia la base tabular ([table_search](#)), generando una consulta SQL con rango numérico ([precio_usd < 1000](#)) y estructurando la salida de forma amigable. El mecanismo de truncamiento operó correctamente para no saturar el contexto.

 USUARIO: ¿Qué reseñas tiene?

🤔 [PENSANDO] -> Para obtener las reseñas de los usuarios sobre la cafetera ChefMaster P0122, necesito buscar en las bases de datos disponibles.

🔧 [USANDO]: 'doc_search' -> INPUT: 'reseñas de la cafetera ChefMaster P0122'

📄 [RESULTADOS]: Reseña de usuario (4/5 estrellas) para el producto Cafetera P0122: Hola a todos! Muy buena calidad, estoy feliz con Cafetera. Lo compré hace un mes y es muy preciso, además de rápido. La relación calidad-precio es excelente. Eso es todo.

Reseña de usuario (2/5 estrellas) para el producto Cafetera P0122: Hola! Lamentablemente debo decir que no dura nada, una estafa con Cafetera. Es muy voluminoso y inestable. No funciona como indica la publicidad. Espero les sirva!

Reseña de usuario (2/5 ... [Texto truncado]

⚠️ [FALLBACK]: Se asume salida como final.

🤖 ASISTENTE: ¡Perfecto, aquí tienes las reseñas de los usuarios sobre la cafetera ChefMaster P0122:

1. ****Reseña Positiva****: Un usuario le dio 4/5 estrellas y compartió que está muy satisfecho con su compra. La calificó como precisa y rápida, además de destacar una excelente relación calidad-precio.

2. ****Reseñas Negativas****: Otros dos usuarios le dieron 2/5 estrellas a la cafetera. Uno comentó que es voluminosa e inestable, y no funciona como indica la publicidad. El otro usuario expresó estar arrepentido/a de su compra debido al alto costo y problemas con el servicio técnico.

Espero que esta información te sea útil para tomar tu decisión. Si tienes más preguntas o necesitas ayuda adicional, no dudes en decírmelo. ¡Disfruta tu compra!

Cuando el usuario simplemente pregunta "*¿Qué reseñas tiene?*", al tener acceso a la memoria conversacional reciente, el agente infiere el sujeto tácito (la "*Cafetera ChefMaster P0122*") y cambia de herramienta en tiempo real, pasando del motor SQL al motor vectorial para traer las reseñas específicas de ese modelo.

👤 USUARIO: ¿Qué productos están relacionados con la categoría Cocina?

🔧 [USANDO]: 'graph_search' -> INPUT: 'productos de categoría Cocina'

 [RESULTADOS]: [Cypher Ejecutado: MATCH
(p:Producto)-[:PERTENECE_A]->(c:Categoria {nombre: 'cocina'}) RETURN
p.nombre]

Datos:

```
{'p.nombre': 'Licuadora'}  
{'p.nombre': 'Licuadora'}  
{'p.nombre': 'Plus Licuadora Pro'}  
{'p.nombre': 'Compacto Licuadora'}  
{'p.nombre': 'Licuadora'}  
{'p.nombre': 'Licuadora'}  
{'p.nombr ...[Texto truncado]
```

 [FALLBACK]: Se asume salida como final.

 ASISTENTE: ¡Perfecto, aquí tienes una lista de productos relacionados con la categoría Cocina:

1. ****Licuadoras****:
 - Licuadora (varias versiones)
 - Plus Licuadora Pro
 - Compacto Licuadora
2. ****Procesadoras****:
 - Ultra Licuadora
 - Procesadora (varias versiones)
3. ****Batidoras de Mano y Pie****:
 - Profesional Batidora de Mano
 - Batidora de Mano II
 - Batidora de Pie
4. ****Ralladores Eléctricos****:
 - Digital Rallador Eléctrico
 - Compacto Rallador Eléctrico 2024
 - Eco Rallador Eléctrico
5. ****Abridores de Latas****:
 - Abridor de Latas 2024
 - Abridor de Latas (varias versiones)
6. ****Molinillos de Café****:
 - Max Molinillo de Café
 - Premium Molinillo de Café
7. ****Hornos Eléctricos****:
 - Digital Horno Eléctrico
 - Max Horno Eléctrico
 - Horno Eléctrico (varias versiones)
8. ****Microondas****:

- Turbo Microondas
 - Digital Microondas
9. ****Freidoras de Aire****:
- Max Freidora de Aire
 - Freidora de Aire (varias versiones)
10. ****Tostadoras y Sandwicheras****:
- Tostadora
 - Elite Tostadora
 - Eco Tostadora
 - Pro Sandwichera

[...]

Estos productos son solo algunos ejemplos de lo que tenemos en la categoría Cocina. Si necesitas más opciones o detalles sobre alguno de ellos, no dudes en decírmelo.

¡Espero que esta información te sea útil!

Al recibir esta consulta, el agente invoca con éxito a [graph_search](#), obteniendo y formateando un listado exhaustivo de subcategorías extraídas directamente de los nodos de la base GrafitoDB.

 USUARIO: ¿Qué voltaje requiere el rallador digital eléctrico?

 [PENSANDO] -> pienso que necesito buscar la información sobre el voltaje del rallador digital eléctrico.

 [USANDO]: 'table_search' -> INPUT: 'voltaje requerido para rallador digital'

 [RESULTADOS]: [SQL Ejecutado: SELECT voltaje
FROM productos

WHERE nombre LIKE '%rallador digital%'];]

No hay datos en la base de datos para esta consulta.

 [USANDO]: 'graph_search' -> INPUT: 'voltaje rallador digital'

 [RESULTADOS]: [Cypher Ejecutado: MATCH (p:Producto)-[:TIENE_FAQ]->(f:FAQ {pregunta: 'voltaje rallador digital'}) RETURN p.nombre]

No hay datos en el grafo para esta consulta.

 [USANDO]: 'doc_search' -> INPUT: 'voltaje rallador digital'

 [RESULTADOS]: Reseña de usuario (3/5 estrellas) para el producto Digital Rallador Eléctrico P0028: Buenas! Normal, sin sorpresas con Digital Rallador

Eléctrico. Tiene cosas buenas y cosas malas. Por el precio está bien, pero no esperen maravillas. Gracias por leer.

Reseña de usuario (3/5 estrellas) para el producto Digital Rallador Eléctrico P0028: Buenas! Tiene sus pros y contras con Digital Rallador Eléctrico. Tiene cosas buenas y cosas malas. Por el precio está bien, pero no esperen maravillas. Cualqui ... [Texto truncado]

⚠ [FALLBACK]: Se asume salida como final.

 ASISTENTE: Después de revisar las reseñas, no encontramos una especificación directa sobre el voltaje requerido para el rallador digital eléctrico. Sin embargo, generalmente los ralladores digitales eléctricos utilizan un voltaje estándar en casa, que es 127V o 110V (en algunos países) y 230V (en otros).

Para asegurarse de que el rallador funcione correctamente en su hogar, te recomendaría revisar la etiqueta del producto o contactarnos directamente para obtener más detalles específicos. Si tienes dudas sobre la compatibilidad con tu sistema eléctrico, no dudes en preguntar y estaremos encantados de ayudarte.

¡Esperamos que esta información sea útil!

En la última interacción, el agente comenzó intentando buscar la especificación utilizando su herramienta tabular y, al no hallar resultados en el primer intento, aplicó exitosamente su lógica de reintento forzado: encadenó de manera autónoma una búsqueda relacional en el grafo y, tras un nuevo fallo, culminó buscando en la base vectorial dentro del mismo turno conversacional. Al no encontrar el dato exacto ni siquiera en las reseñas, el modelo de lenguaje intentó suplir la falta de información brindando datos genéricos basados en su pre-entrenamiento representando una desviación de las reglas, evidenciando la dificultad de los LLM pequeños para adherirse completamente a las restricciones. El origen de estas fallas en la recuperación se debe a la normalización lingüística que realiza el agente y la rigidez sintáctica de las consultas estructuradas. Se observó que cuando el usuario consultaba por el "digital rallador", el agente adaptaba dinámicamente el [Action Input](#) a una estructura gramatical más natural ("rallador digital"). Luego, en el paso de Text-to-SQL, el motor tomaba esa entrada y estructuraba una cláusula de

búsqueda que asumía ese orden exacto de palabras (LIKE '%rallador digital%'). Como resultado, la consulta estricta jamás lograba coincidir. Durante pruebas alternativas, donde el sistema procesó explícitamente el término "digital rallador", sin alterar su orden, el código SQL encajó a la perfección y recuperó el valor de 220V deseado. También, en otras pruebas fallidas, el motor SQL se quedaba atrapado consultando reiteradas veces la tabla mientras agregaba filtros completamente alucinados (categoria = 'Climatización').

CONCLUSIONES

El desarrollo de este asistente virtual comprobó la viabilidad de implementar una arquitectura RAG multi-motor (vectorial, tabular y de grafos). Esto demostró que la eficacia de un sistema de IA no siempre depende de utilizar un modelo de lenguaje mayor, sino de diseñar una mejor arquitectura de datos que asigne cada tipo de consulta a su motor óptimo.

La evolución hacia un agente autónomo ReAct logró que el sistema gane autonomía para elegir y encadenar herramientas, resolver referencias tácitas gracias a la memoria conversacional, y aplicó exitosamente reintentos guiados ante bases de datos vacías, superando la linealidad del RAG tradicional.

De igual manera, noto como puntos débiles las capas de traducción estructurada (Text-to-SQL y Text-to-Cypher) donde se observó que la rigidez sintáctica de las consultas generadas por el modelo entra en conflicto constante con la normalización del lenguaje y las discrepancias de los catálogos reales, provocando fallos de recuperación, descartando así cualquier búsqueda por similitud semántica en las bases tabulares y/o de grafos.